

PostgreSQL on Windows — Installation Guide

Beginner's Guide

2025-10-27

This guide walks you through step-by-step: installing PostgreSQL, verifying your setup, using the `psql` console, loading the Pagila sample database, and using beginner-friendly GUI clients. Clear tips and quick solutions are included throughout.

Before You Start

Key Terms (Quick Reference) - **Server** — the PostgreSQL database engine running on your PC - **Client** — tools you use to communicate with the server (e.g., `psql`, pgAdmin, DBeaver) - **Database** — a container for tables, views, etc. (one server can host multiple databases) - **Role/User** — an account that logs in and has permissions

Installing PostgreSQL (Windows)

Download PostgreSQL

Official Link: <https://www.postgresql.org/download/windows/>

1. Open the **official PostgreSQL for Windows** page and choose **Interactive installer by EDB**
2. Download the latest stable version for **Windows x86-64**
3. Run the installer (`.exe`) and keep the default components: **Server, pgAdmin, Command Line Tools**
4. When prompted, **set a password for the superuser postgres** and keep it safe
5. Keep the **Port** at 5432 and accept the default **Locale**
6. Complete the installation wizard

Installed Components - PostgreSQL Server — the engine that stores your data

- **psql** — the command-line SQL shell
- **pgAdmin** — the official graphical management tool

Verify Installation (Choose One)

Option A — Start Menu - Start Menu → **SQL Shell (psql)** → press **Enter** for each prompt → enter your password. If you see **postgres=#**, you're good.

Option B — Command Prompt (CMD)

```
"C:\Program Files\PostgreSQL\17\bin\psql.exe" --version
```

(replace 17 with your installed version)

Expected Output:

```
psql (PostgreSQL) 17.0
```

If Windows Says “psql is not recognized”

- Use **Start** → **SQL Shell (psql)** (works without PATH), or
- Add PostgreSQL's bin folder to your **PATH**:
 - System → About → Advanced system settings
 - Environment Variables → Path → Edit → New
 - Add: C:\Program Files\PostgreSQL\17\bin
 - Then reopen your terminal

Starting/Stopping the Database Service (If Needed)

- Press **Win + R**, type **services.msc**, press **Enter**
- Find **PostgreSQL** . Right-click → **Start** (or **Restart/Stop**). It normally starts automatically with Windows.

If you see connection timeouts or “could not connect to server”, check this service first.

First Connection with psql

1. Open **SQL Shell (psql)** from the Start menu
2. Press **Enter** at each prompt to accept defaults (Server localhost, Database postgres, Port 5432, User postgres)
3. Enter your **postgres password**. You should see a prompt like `postgres=#`

What is psql? (Gentle Introduction)

- **psql** is a text console for communicating with PostgreSQL. You type a command, press **Enter**, and the server responds
- **Why learn it?** It's fast, matches official documentation, and helps you troubleshoot even when a GUI can't connect
- **What you'll see:** a prompt like `postgres=#` (or your database name). End SQL statements with a **semicolon (;)**. If you forget it, **psql** will wait for more input
- **Two types of commands:**
 - **SQL** (ends with `;`) — e.g., `SELECT version();`
 - **Meta-commands** (start with backslash) — e.g., `\dt` (list tables), `\d film` (describe a table). These are **psql** features, not SQL
- **Mini Demo:**

```
SELECT 1;
CREATE DATABASE testdb;
\c testdb
CREATE TABLE t(x int);
INSERT INTO t VALUES (1),(2);
SELECT * FROM t;
DROP TABLE t;
\dt
```

- **Common Beginner Mistakes:** missing semicolon; connecting to wrong database; doing everything as superuser **postgres** (create a normal user for daily work)

Analogy: PostgreSQL is the **library** (server). **psql** is the **librarian's desk**: you ask for things (SQL) and get answers. GUIs add maps and buttons, but the desk still works.

More psql Examples

Example 1: Checking Server Version

```
SELECT version();
```

Example 2: Creating and Switching Databases

```
-- Create a new database
CREATE DATABASE myproject;

-- List all databases
\l

-- Connect to your new database
\c myproject

-- Check current database
SELECT current_database();
```

Example 3: Creating Your First Table

```
-- Create a simple table
CREATE TABLE employees (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    department VARCHAR(50),
    salary NUMERIC(10, 2)
);

-- Describe the table structure
\d employees

-- Insert some data
INSERT INTO employees (name, department, salary) VALUES
    ('Alice Johnson', 'Engineering', 75000),
    ('Bob Smith', 'Marketing', 65000),
    ('Carol White', 'Engineering', 80000);

-- Query the data
SELECT * FROM employees;
```

```
-- Query with filtering
SELECT name, salary FROM employees WHERE department = 'Engineering';
```

Example 4: Basic Aggregations

```
-- Count employees by department
SELECT department, COUNT(*) as employee_count
FROM employees
GROUP BY department;

-- Average salary by department
SELECT department, AVG(salary) as avg_salary
FROM employees
GROUP BY department
ORDER BY avg_salary DESC;
```

Example 5: Updating and Deleting Data

```
-- Update a record
UPDATE employees
SET salary = 85000
WHERE name = 'Alice Johnson';

-- Verify the update
SELECT * FROM employees WHERE name = 'Alice Johnson';

-- Delete a record (be careful!)
DELETE FROM employees WHERE name = 'Bob Smith';

-- See what's left
SELECT * FROM employees;
```

Loading the Pagila Sample Database

Pagila is a classic example (DVD rental store) with realistic tables (films, actors, customers, rentals). It's perfect for practicing joins, aggregates, and foreign keys.

Download the Files

Official Pagila Link: <https://github.com/devrimgunduz/pagila>

Create folder C:\Pagila_Project and download these two files:

- **Schema:** [pagila-schema.sql](#)
- **Data:** [pagila-data.sql](#)

(Right-click links → Save As... → Save to C:\Pagila_Project)

Do **not** edit these official Pagila SQL files. Save them exactly as downloaded:

- C:\Pagila_Project\pagila-schema.sql
- C:\Pagila_Project\pagila-data.sql

Create and Import the Database as postgres

Open psql as the admin user postgres, then run the commands in this order:

```
CREATE DATABASE pagila;  
\c pagila  
\i 'C:/Pagila_Project/pagila-schema.sql'  
\i 'C:/Pagila_Project/pagila-data.sql'
```

Note: Use forward slashes (/) in psql paths, even on Windows. Import the schema first, then the data.

Alternative Import from Command Prompt (CMD) or PowerShell

For CMD:

```
cd C:\Pagila_Project  
set PSQL="C:\Program Files\PostgreSQL\17\bin\psql.exe"  
%PSQL% -U postgres -d pagila -f pagila-schema.sql  
%PSQL% -U postgres -d pagila -f pagila-data.sql
```

For PowerShell:

```
cd C:\Pagila_Project  
$PSQL = "C:\Program Files\PostgreSQL\17\bin\psql.exe"  
& $PSQL -U postgres -d pagila -f pagila-schema.sql  
& $PSQL -U postgres -d pagila -f pagila-data.sql
```

(Replace 17 with your installed version.)

Create Your Personal User

After the import succeeds, stay connected as `postgres` and create a normal user for daily TP work. In the example below, `karim` is a Moroccan first-name example.

```
CREATE ROLE karim WITH LOGIN PASSWORD 'votre_mot_de_passe';
GRANT ALL PRIVILEGES ON DATABASE pagila TO karim;
GRANT ALL ON SCHEMA public TO karim;
GRANT ALL ON ALL TABLES IN SCHEMA public TO karim;
GRANT ALL ON ALL SEQUENCES IN SCHEMA public TO karim;
```

Each student replaces `karim` with their own lowercase first name, for example `amina`, `youssef`, or `salma`. Choose a private password and do not share it.

To connect directly from CMD with your personal user, run:

```
"C:\Program Files\PostgreSQL\<version>\bin\psql.exe" -h localhost -p 5432 -U
karim -d pagila
```

The same command split across two lines for readability:

```
"C:\Program Files\PostgreSQL\<version>\bin\psql.exe" ^
-h localhost -p 5432 -U karim -d pagila
```

Confirm It Worked

```
\c pagila
\dt
SELECT COUNT(*) FROM film;           -- ~1,000
SELECT COUNT(*) FROM customer;       -- ~599
SELECT COUNT(*) FROM actor;          -- ~200
```

Prefer a GUI? Popular Clients

Choose **one** to start; you can always try others later. (Beginners often choose **pgAdmin** or **DBeaver**.)

A) pgAdmin (Official)

Download: <https://www.pgadmin.org/download/>

- **Best for:** Administration + learning core PostgreSQL concepts
- **Connection:** Open pgAdmin → Register → Server... → Host `localhost`, Port `5432`, Database `pagila`, User `karim` (or your own lowercase first name), password → **Save**
- **Usage:** Create databases/roles, browse, run queries in **Query Tool**

Example: Running Your First Query in pgAdmin 1. Expand Servers → PostgreSQL 17 → Databases → `pagila` 2. Right-click `pagila` → Query Tool 3. Type: `SELECT * FROM film LIMIT 10;` 4. Click the lightning bolt icon (Execute) or press F5

B) DBeaver Community (Free, Multi-DB)

Download: <https://dbeaver.io/download/>

- **Best for:** Daily queries and data exploration; works with many databases
- **Connection:** New Database Connection → PostgreSQL → Host `localhost`, Port `5432`, Database `pagila` (or `postgres`), User + Password → **Test** → **Finish**
- **Cool Features:** ER diagrams, data editors, CSV/Excel import/export

Example: Exploring Data in DBeaver 1. Connect to your `pagila` database 2. Expand `pagila` → Schemas → `public` → Tables 3. Right-click `film` table → View Data 4. Use the filter row at the top to search (e.g., `rating = 'PG'`) 5. Export results: right-click grid → Export Data

C) Visual Studio Code + PostgreSQL Extension (Free)

Download: <https://marketplace.visualstudio.com/items?itemName=ms-ossdata.vscode-postgresql>

- **Best for:** Developers who live in VS Code
- **Connection:** Extensions → search **PostgreSQL** (Microsoft) → Install → Command Palette → **PostgreSQL: Add Connection** → fill `host/port/database/user/password`
- **Cool Features:** Run queries next to your code; IntelliSense; results grid

Example: Quick Query in VS Code 1. Create new file: `queries.sql` 2. Connect to database using Command Palette (Ctrl+Shift+P): PostgreSQL: Connect 3. Write your query: `SELECT title, rating FROM film WHERE rating = 'PG' LIMIT 5;` 4. Highlight the query text 5. Right-click → Execute Query (or use keyboard shortcut)

Which GUI to Choose? (Quick Picks)

- Brand new to databases? **pgAdmin** (official, safe default choice)
- Modern, user-friendly explorer? **DBeaver Community**
- I live in VS Code → **PostgreSQL for VS Code** extension

How to Connect (Same Settings in Most GUIs)

- **Host:** localhost
- **Port:** 5432 (unless you changed it)
- **Database:** **pagila** (after creation) or **postgres** to start
- **User:** **karim** (or your own lowercase first name); use **postgres** only for administration
- **Password:** what you set during installation or for that role

Pro Tips

- Save the connection so you don't retype credentials
- Create a second connection for your normal user (not **postgres**) to practice least privilege
- Learn your client's **export to CSV/Excel** feature for homework and lab reports

Common GUI Pitfalls

- **Service not started:** Launch **Services** → *PostgreSQL* → **Start/Restart**
- **Wrong host/port:** Default host **localhost**, port **5432**. If *localhost* fails, try **127.0.0.1**
- **Wrong credentials/database:** Ensure the **user** and **database** exist; test with **psql** first
- **Saved/stale password:** Update saved password in your GUI
- **SSL prompts (local):** For local work, SSL is usually unnecessary — set mode to **Prefer** (or **Disable** if allowed) unless your setup requires SSL

Troubleshooting GUI Connection — Quick Flow

1. Is PostgreSQL running? **Services** → **PostgreSQL <version>** should be **Running**.
2. Does the server respond?

CMD:

```
"%PGBIN%\pg_isready.exe" -h localhost -p 5432
```

PowerShell:

```
& "$PGBIN\pg_isready.exe" -h localhost -p 5432
```

3. Can you connect with psql?

CMD:

```
"%PGBIN%\psql.exe" -U postgres -d postgres -c "SELECT 1;"
```

PowerShell:

```
& "$PGBIN\psql.exe" -U postgres -d postgres -c "SELECT 1;"
```

4. Is the port listening?

CMD:

```
netstat -ano | findstr 5432
```

PowerShell:

```
Get-NetTCPConnection -LocalPort 5432
```

5. Try host variations: 127.0.0.1 instead of localhost (IPv4 vs IPv6).
6. Reset/create user if needed:

```
ALTER ROLE postgres PASSWORD 'NewStrongPassword!';
```

7. Firewall/security tools: allow postgres.exe on port 5432 (private network).

Remote Connections (Later): For another computer to connect to your server, you'll need to adjust postgresql.conf (listen_addresses) and pg_hba.conf. For this beginner guide (same PC), no changes required.

Useful Windows Commands (Copy/Paste)

For CMD (Command Prompt):

First set the path:

```
set PGBIN=C:\Program Files\PostgreSQL\17\bin
```

Then use:

```
"%PGBIN%\psql.exe" --version  
"%PGBIN%\pg_isready.exe" -h localhost -p 5432  
"%PGBIN%\psql.exe" -U postgres -d postgres  
-c "SELECT current_database();"
```

For PowerShell:

First set the path:

```
$PGBIN = "C:\Program Files\PostgreSQL\17\bin"
```

Then use:

```
& "$PGBIN\psql.exe" --version  
 & "$PGBIN\pg_isready.exe" -h localhost -p 5432  
 & "$PGBIN\psql.exe" -U postgres -d postgres  
-c "SELECT current_database();"
```

psql Essentials (Cheat Sheet)

<code>\q</code>	-- quit psql
<code>\l</code>	-- list databases
<code>\c dbname</code>	-- connect to a database (e.g., <code>\c pagila</code>)
<code>\dt</code>	-- list tables in current schema
<code>\d table_name</code>	-- describe a table (e.g., <code>\d film</code>)
<code>\du</code>	-- list roles/users
<code>\timing</code>	-- toggle query timing on/off
<code>\i path.sql</code>	-- execute commands from a file
<code>\?</code>	-- help for meta-commands
<code>\h</code>	-- help for SQL commands
<code>\h SELECT</code>	-- help for specific SQL command

More Useful Meta-Commands:

```
\dn          -- list schemas
\df          -- list functions
\dv          -- list views
\x           -- toggle expanded display (better for wide rows)
\e           -- open text editor to write/edit query
\g           -- execute previous query again
\s           -- display command history
```

Your First Queries (Pagila)

Connect to pagila:

```
\c pagila
```

Warm-up Queries (Simple)

```
-- How many films?
SELECT COUNT(*) FROM film;

-- See a few films
SELECT * FROM film LIMIT 5;

-- G-rated films
SELECT title, rating FROM film WHERE rating = 'G' LIMIT 10;

-- Longest films
SELECT title, length FROM film ORDER BY length DESC LIMIT 10;

-- Films with "LOVE" in the title
SELECT title, release_year FROM film WHERE title LIKE '%LOVE%';
```

Intermediate Queries

```

-- Top 10 most expensive PG films to rent
SELECT title, rental_rate
FROM film
WHERE rating = 'PG'
ORDER BY rental_rate DESC, title
LIMIT 10;

-- Average film length by rating
SELECT rating,
       ROUND(AVG(length), 2) as avg_length,
       COUNT(*) as film_count
FROM film
GROUP BY rating
ORDER BY avg_length DESC;

-- Films with above-average rental rates
SELECT title, rental_rate
FROM film
WHERE rental_rate > (SELECT AVG(rental_rate) FROM film)
ORDER BY rental_rate DESC
LIMIT 15;

-- Count films by category
SELECT c.name as category, COUNT(*) as film_count
FROM category c
JOIN film_category fc ON c.category_id = fc.category_id
GROUP BY c.name
ORDER BY film_count DESC;

```

Advanced Queries to Explore

```

-- Rentals per customer (top 10)
SELECT c.customer_id,
       c.first_name || ' ' || c.last_name AS customer,
       COUNT(*) AS rentals
FROM rental r
JOIN customer c USING (customer_id)
GROUP BY c.customer_id, customer
ORDER BY rentals DESC
LIMIT 10;

```

```

-- Revenue by category
SELECT ca.name AS category,
       SUM(p.amount) AS total_revenue
FROM payment p
JOIN rental r USING (rental_id)
JOIN inventory i USING (inventory_id)
JOIN film f USING (film_id)
JOIN film_category fc USING (film_id)
JOIN category ca USING (category_id)
GROUP BY ca.name
ORDER BY total_revenue DESC;

-- Most popular actors (by number of films)
SELECT a.actor_id,
       a.first_name || ' ' || a.last_name AS actor_name,
       COUNT(fa.film_id) AS film_count
FROM actor a
JOIN film_actor fa USING (actor_id)
GROUP BY a.actor_id, actor_name
ORDER BY film_count DESC
LIMIT 10;

-- Films starring a specific actor
SELECT f.title, f.release_year, f.rating
FROM film f
JOIN film_actor fa USING (film_id)
JOIN actor a USING (actor_id)
WHERE a.first_name = 'PENELOPE' AND a.last_name = 'GUINNESS'
ORDER BY f.title;

-- Customers who haven't rented in the last 30 days
SELECT c.customer_id,
       c.first_name || ' ' || c.last_name AS customer,
       MAX(r.rental_date) AS last_rental
FROM customer c
LEFT JOIN rental r USING (customer_id)
GROUP BY c.customer_id, customer
HAVING MAX(r.rental_date) < CURRENT_DATE - INTERVAL '30 days'
      OR MAX(r.rental_date) IS NULL
ORDER BY last_rental NULLS FIRST;

-- Monthly revenue trend

```

```

SELECT DATE_TRUNC('month', payment_date) AS month,
       SUM(amount) AS monthly_revenue,
       COUNT(*) AS transaction_count
FROM payment
GROUP BY month
ORDER BY month;

```

Window Function Examples

```

-- Rank films by rental rate within each rating
SELECT title, rating, rental_rate,
       RANK() OVER (PARTITION BY rating ORDER BY rental_rate DESC) as rank_in_rating
FROM film
ORDER BY rating, rank_in_rating
LIMIT 20;

-- Running total of payments
SELECT payment_date,
       amount,
       SUM(amount) OVER (ORDER BY payment_date) as running_total
FROM payment
ORDER BY payment_date
LIMIT 50;

-- Compare each film's length to category average
SELECT f.title, c.name as category, f.length,
       ROUND(AVG(f.length) OVER (PARTITION BY c.category_id), 2) as category_avg_length,
       f.length - ROUND(AVG(f.length) OVER (PARTITION BY c.category_id), 2) as diff_from_avg
FROM film f
JOIN film_category fc USING (film_id)
JOIN category c USING (category_id)
ORDER BY category, diff_from_avg DESC;

```

Troubleshooting (Quick Reference)

"psql is not recognized ..."

- Use Start → SQL Shell (psql) (no PATH needed), or add

C:\Program Files\PostgreSQL\<version>\bin to PATH; reopen terminal.

"password authentication failed for user 'postgres'"

- Retype carefully (case-sensitive). If you can connect via pgAdmin, change password there. Completely stuck on fresh install? Reinstall may be fastest.

"could not connect to server" / timeouts

- Ensure PostgreSQL service is running (Services). Check host localhost and port 5432.

"Port already in use"

- Another app is using 5432. Stop it, or install PostgreSQL on a different port (e.g., 5433) and use that port in clients.

"relation does not exist"

- Check you're connected to the right database (\c dbname)
- Verify table name spelling (case-sensitive if quoted)
- Use \dt to list available tables

"syntax error at or near..."

- Check for missing semicolons
- Verify SQL keyword spelling
- Check for unmatched quotes or parentheses

"OWNER TO student" appears in a Pagila schema file

- That is a course-modified file, not the official Pagila file.
- Download the official `pagila-schema.sql` again instead of editing it.

Security Best Practices

Your Personal Non-Superuser for Practice

Instead of always using `postgres`, create a normal user after importing Pagila. Replace `karim` with your own lowercase first name, and keep your password private:

```
CREATE ROLE karim WITH LOGIN PASSWORD 'votre_mot_de_passe';
GRANT ALL PRIVILEGES ON DATABASE pagila TO karim;
```



```
-- Also grant schema privileges
\c pagila
GRANT ALL ON SCHEMA public TO karim;
GRANT ALL ON ALL TABLES IN SCHEMA public TO karim;
GRANT ALL ON ALL SEQUENCES IN SCHEMA public TO karim;

-- Then connect with this user
\c pagila karim
```

Why? Principle of least privilege: use `postgres` only for administration, not daily queries.

Example: Creating Department-Specific Users

```
-- Read-only analyst
CREATE ROLE analyst WITH LOGIN PASSWORD 'analyst_pass';
GRANT CONNECT ON DATABASE pagila TO analyst;
GRANT USAGE ON SCHEMA public TO analyst;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO analyst;

-- Developer with write access
CREATE ROLE developer WITH LOGIN PASSWORD 'dev_pass';
GRANT CONNECT ON DATABASE pagila TO developer;
GRANT USAGE ON SCHEMA public TO developer;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO developer;

-- Manager who can create tables
CREATE ROLE manager WITH LOGIN PASSWORD 'manager_pass';
GRANT CONNECT ON DATABASE pagila TO manager;
GRANT ALL ON SCHEMA public TO manager;
```

Common Student Mistakes

- Forgetting semicolons in SQL statements
- Using Windows backslashes (\) in file paths in `psql` (use /)
- Staying connected to wrong database (use `\c` to switch)
- Running DDL commands without thinking (DROP, DELETE without WHERE)
- Mixing SQL and meta-commands (meta-commands don't need ;)

- **Not checking which database you're in** before creating tables
 - **Forgetting quotes** around string values in WHERE clauses
 - **Case sensitivity issues** with table/column names when using quotes
-

What's Next? Next Steps

Important Concepts to Learn Next

- **Indexes** — speed up queries
- **Transactions** — ensure data consistency
- **Constraints** — primary keys, foreign keys, checks
- **Views** — saved queries
- **Functions and Procedures** — reusable logic
- **Triggers** — automatic actions on data changes
- **JSON support** — storing and querying JSON data

Recommended Resources

- **Official PostgreSQL Documentation:** <https://www.postgresql.org/docs/>
- **PostgreSQL Exercises (interactive practice):** <https://pgexercises.com/>
- **PostgreSQL Tutorial:** <https://www.postgresqtutorial.com/>
- **Pagila GitHub (more examples):** <https://github.com/devringunduz/pagila>

Practice Exercises with Pagila

1. Find all films featuring actor “PENELOPE GUINESS”
2. Calculate average rental duration by film category
3. Identify customers who have never rented a film
4. Create a view showing films in stock at each store
5. Write a query to find the most profitable films
6. Find which day of the week has the most rentals
7. List films that have never been rented
8. Calculate the lifetime value of each customer
9. Find actors who appear in both Action and Comedy films
10. Identify the store with higher revenue

Before Asking for Help — Checklist

- Is the PostgreSQL service running?
 - Have you tried connecting with `psql`?
 - Are you connected to the right database?
 - Have you checked spelling (tables, columns, passwords)?
 - Have you reviewed the complete error messages?
 - Have you searched the error in documentation or online?
 - Have you tried the simplest possible query first?
-

Backup and Restore (Basics)

Backup a Database

CMD:

```
set PGBIN=C:\Program Files\PostgreSQL\17\bin
"%PGBIN%\pg_dump.exe" -U postgres -d pagila -f C:\backup_pagila.sql
```

PowerShell:

```
$PGBIN = "C:\Program Files\PostgreSQL\17\bin"
& "$PGBIN\pg_dump.exe" -U postgres -d pagila -f C:\backup_pagila.sql
```

Backup with Custom Format (Compressed, Faster Restore)

CMD:

```
"%PGBIN%\pg_dump.exe" -U postgres -d pagila -Fc -f C:\backup_pagila.dump
```

PowerShell:

```
& "$PGBIN\pg_dump.exe" -U postgres -d pagila -Fc -f C:\backup_pagila.dump
```

Restore a Database

From SQL file (CMD):

```
"%PGBIN%\psql.exe" -U postgres -d pagila -f C:\backup_pagila.sql
```

From SQL file (PowerShell):

```
& "$PGBIN\psql.exe" -U postgres -d pagila -f C:\backup_pagila.sql
```

From custom format (CMD):

```
"%PGBIN%\pg_restore.exe" -U postgres -d pagila C:\backup_pagila.dump
```

From custom format (PowerShell):

```
& "$PGBIN\pg_restore.exe" -U postgres -d pagila C:\backup_pagila.dump
```

Tip: Back up your practice databases regularly before experimenting with destructive commands!

Automated Backup Script Example

PowerShell script (save as backup_postgres.ps1):

```
$PGBIN = "C:\Program Files\PostgreSQL\17\bin"
$BACKUP_DIR = "C:\PostgreSQL_Backups"
$DATE = Get-Date -Format "yyyy-MM-dd_HH:mm:ss"

# Create backup directory if it doesn't exist
if (-not (Test-Path $BACKUP_DIR)) {
    New-Item -ItemType Directory -Path $BACKUP_DIR
}

# Backup pagila database
& "$PGBIN\pg_dump.exe" -U postgres -d pagila -Fc -f "$BACKUP_DIR\pagila_$DATE.dump"

Write-Host "Backup completed: pagila_$DATE.dump"
```

You're Ready!

Practice a little each day — with `psql` or your favorite GUI — and you'll feel comfortable quickly. Explore schemas and keys, try `EXPLAIN ANALYZE`, and keep notes of commands you run. Happy learning and happy querying!

Remember: **consistency beats intensity**. 15 minutes daily is better than 2 hours once a week!